We note that the domain relational calculus also does not have any equivalent of the aggregate operation, but it can be extended to support aggregation, and extending it to handle arithmatic expressions is straightforward.

3.8 Summary

- The **relational data model** is based on a collection of tables. The user of the database system may query these tables, insert new tuples, delete tuples, and update (modify) tuples. There are several languages for expressing these operations.
- The **relational algebra** defines a set of algebraic operations that operate on tables, and output tables as their results. These operations can be combined to get expressions that express desired queries. The algebra defines the basic operations used within relational query languages.
- The operations in relational algebra can be divided into
  - Basic operations
  - Additional operations that can be expressed in terms of the basic operations
  - Extended operations, some of which add further expressive power to relational algebra
- Databases can be modified by **insertion**, **deletion**, or **update** of tuples. We used the relational algebra with the **assignment operator** to express these modifications.
- Different users of a shared database may benefit from individualized **views** of the database. Views are "virtual relations" defined by a query expression. We evaluate queries involving views by replacing the view with the expression that defines the view.
- Views are useful mechanisms for simplifying database queries, but modification of the database through views may cause problems. Therefore, database systems severely restrict updates through views.
- For reasons of query-processing efficiency, a view may be **materialized**—that is, the query is evaluated and the result stored physically. When database relations are updated, the materialized view must be correspondingly updated.
- The **tuple relational calculus** and the **domain relational calculus** are nonprocedural languages that represent the basic power required in a relational query language. The basic relational algebra is a procedural language that is equivalent in power to both forms of the relational calculus when they are restricted to safe expressions.
- The relational algebra and the relational calculi are terse, formal languages that are inappropriate for casual users of a database system. Commercial database systems, therefore, use languages with more "syntactic sugar." In Chap-

ters 4 and 5, we shall consider the three most influential languages: **SQL**, which is based on relational algebra, and **QBE** and **Datalog**, which are based on domain relational calculus.

Review Terms

- Table
- Relation
- Tuple variable
- Atomic domain
- Null value
- Database schema
- Database instance
- Relation schema
- Relation instance
- Keys
- Foreign key
  - Referencing relation
  - Referenced relation
- Schema diagram
- Query language
- Procedural language
- Nonprocedural language
- Relational algebra
- Relational algebra operations
  - Select σ
  - Project Π
  - Union $\cup$
  - Set difference $-$
  - Cartesian product $\times$
  - Rename ρ
- Additional operations
  - Set-intersection $\cap$
  - Natural-join $\bowtie$
  - Division /
- Assignment operation
- Extended relational-algebra operations
  - Generalized projection Π
  - Outer join
    - Left outer join $\supset\!\bowtie$
    - Right outer join $\bowtie\!\subset$
    - Full outer join $\supset\!\bowtie\!\subset$
  - Aggregation $\mathcal{G}$
- Multisets
- Grouping
- Null values
- Modification of the database
  - Deletion
  - Insertion
  - Updating
- Views
- View definition
- Materialized views
- View update
- View expansion
- Recursive views
- Tuple relational calculus
- Domain relational calculus
- Safety of expressions
- Expressive power of languages

Exercises

3.1 Design a relational database for a university registrar's office. The office maintains data about each class, including the instructor, the number of students enrolled, and the time and place of the class meetings. For each student–class pair, a grade is recorded.

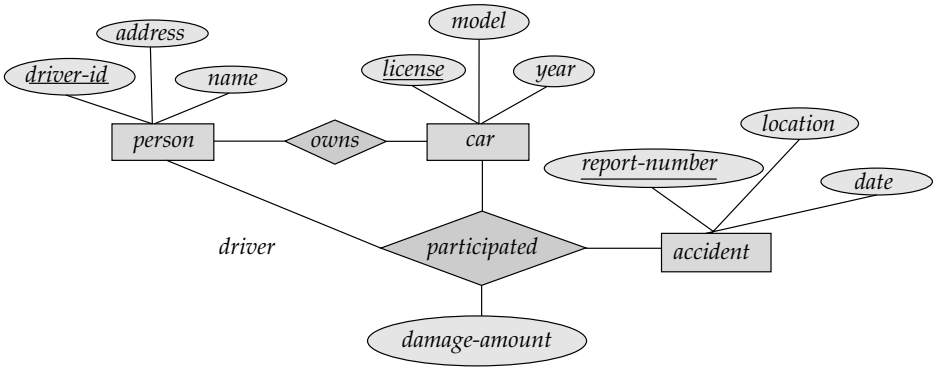


Figure 3.38 E-R diagram.

3.2 Describe the differences in meaning between the terms *relation* and *relation schema*. Illustrate your answer by referring to your solution to Exercise 3.1.

3.3 Design a relational database corresponding to the E-R diagram of Figure 3.38.

3.4 In Chapter 2, we saw how to represent many-to-many, many-to-one, one-to-many, and one-to-one relationship sets. Explain how primary keys help us to represent such relationship sets in the relational model.

3.5 Consider the relational database of Figure 3.39, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

- **a.** Find the names of all employees who work for First Bank Corporation.
- **b.** Find the names and cities of residence of all employees who work for First Bank Corporation.
- **c.** Find the names, street address, and cities of residence of all employees who work for First Bank Corporation and earn more than $10,000 per annum.
- **d.** Find the names of all employees in this database who live in the same city as the company for which they work.
- **e.** Find the names of all employees who live in the same city and on the same street as do their managers.
- **f.** Find the names of all employees in this database who do not work for First Bank Corporation.
- **g.** Find the names of all employees who earn more than every employee of Small Bank Corporation.
- **h.** Assume the companies may be located in several cities. Find all companies located in every city in which Small Bank Corporation is located.

3.6 Consider the relation of Figure 3.21, which shows the result of the query "Find the names of all customers who have a loan at the bank." Rewrite the query to include not only the name, but also the city of residence for each customer. Observe that now customer Jackson no longer appears in the result, even though Jackson does in fact have a loan from the bank.

employee (*person-name*, *street*, *city*)
works (*person-name*, *company-name*, *salary*)
company (*company-name*, *city*)
manages (*person-name*, *manager-name*)

Figure 3.39 Relational database for Exercises 3.5, 3.8 and 3.10.

a. Explain why Jackson does not appear in the result.
b. Suppose that you want Jackson to appear in the result. How would you modify the database to achieve this effect?
c. Again, suppose that you want Jackson to appear in the result. Write a query using an outer join that accomplishes this desire without your having to modify the database.

3.7 The outer-join operations extend the natural-join operation so that tuples from the participating relations are not lost in the result of the join. Describe how the theta join operation can be extended so that tuples from the left, right, or both relations are not lost from the result of a theta join.

3.8 Consider the relational database of Figure 3.39. Give an expression in the relational algebra for each request:

a. Modify the database so that Jones now lives in Newtown.
b. Give all employees of First Bank Corporation a 10 percent salary raise.
c. Give all managers in this database a 10 percent salary raise.
d. Give all managers in this database a 10 percent salary raise, unless the salary would be greater than $100,000. In such cases, give only a 3 percent raise.
e. Delete all tuples in the *works* relation for employees of Small Bank Corporation.

3.9 Using the bank example, write relational-algebra queries to find the accounts held by more than two customers in the following ways:

a. Using an aggregate function.
b. Without using any aggregate functions.

3.10 Consider the relational database of Figure 3.39. Give a relational-algebra expression for each of the following queries:

a. Find the company with the most employees.
b. Find the company with the smallest payroll.
c. Find those companies whose employees earn a higher salary, on average, than the average salary at First Bank Corporation.

3.11 List two reasons why we may choose to define a view.

3.12 List two major problems with processing update operations expressed in terms of views.

3.13 Let the following relation schemas be given:

$$R = (A, B, C)$$

$$S = (D, E, F)$$

Let relations $r(R)$ and $s(S)$ be given. Give an expression in the tuple relational calculus that is equivalent to each of the following:

a. $\Pi_A(r)$
b. $\sigma_{B=17}(r)$
c. $r \times s$
d. $\Pi_{A,F}(\sigma_{C=D}(r \times s))$

3.14 Let $R = (A, B, C)$, and let r_1 and r_2 both be relations on schema R. Give an expression in the domain relational calculus that is equivalent to each of the following:

a. $\Pi_A(r_1)$
b. $\sigma_{B=17}(r_1)$
c. $r_1 \cup r_2$
d. $r_1 \cap r_2$
e. $r_1 - r_2$
f. $\Pi_{A,B}(r_1) \bowtie \Pi_{B,C}(r_2)$

3.15 Repeat Exercise 3.5 using the tuple relational calculus and the domain relational calculus.

3.16 Let $R = (A, B)$ and $S = (A, C)$, and let $r(R)$ and $s(S)$ be relations. Write relational-algebra expressions equivalent to the following domain-relational-calculus expressions:

a. $\{< a > \mid \exists\, b\, (< a, b > \in r \wedge b = 17)\}$
b. $\{< a, b, c > \mid < a, b > \in r \wedge < a, c > \in s\}$
c. $\{< a > \mid \exists\, b\, (< a, b > \in r) \vee \forall\, c\, (\exists\, d\, (< d, c > \in s) \Rightarrow < a, c > \in s)\}$
d. $\{< a > \mid \exists\, c\, (< a, c > \in s \wedge \exists\, b_1, b_2\, (< a, b_1 > \in r \wedge < c, b_2 > \in r \wedge b_1 > b_2))\}$

3.17 Let $R = (A, B)$ and $S = (A, C)$, and let $r(R)$ and $s(S)$ be relations. Using the special constant *null*, write tuple-relational-calculus expressions equivalent to each of the following:

a. $r \ltimes s$
b. $r \rtimes s$
c. $r \rtimes\!\ltimes s$

3.18 List two reasons why null values might be introduced into the database.

3.19 Certain systems allow *marked* nulls. A marked null $\perp_i$ is equal to itself, but if $i \neq j$, then $\perp_i \neq \perp_j$. One application of marked nulls is to allow certain updates through views. Consider the view *loan-info* (Section 3.5). Show how you can use marked nulls to allow the insertion of the tuple ("Johnson", 1900) through *loan-info*.

Bibliographical Notes

E. F. Codd of the IBM San Jose Research Laboratory proposed the relational model in the late 1960s; Codd [1970]. This work led to the prestigious ACM Turing Award to Codd in 1981; Codd [1982].

After Codd published his original paper, several research projects were formed with the goal of constructing practical relational database systems, including System R at the IBM San Jose Research Laboratory, Ingres at the University of California at Berkeley, Query-by-Example at the IBM T. J. Watson Research Center, and the Peterlee Relational Test Vehicle (PRTV) at the IBM Scientific Center in Peterlee, United Kingdom. System R is discussed in Astrahan et al. [1976], Astrahan et al. [1979], and Chamberlin et al. [1981]. Ingres is discussed in Stonebraker [1980], Stonebraker [1986b], and Stonebraker et al. [1976]. Query-by-example is described in Zloof [1977]. PRTV is described in Todd [1976].

Many relational-database products are now commercially available. These include IBM's DB2, Ingres, Oracle, Sybase, Informix, and Microsoft SQL Server. Database products for personal computers include Microsoft Access, dBase, and FoxPro. Information about the products can be found in their respective manuals.

General discussion of the relational data model appears in most database texts. Atzeni and Antonellis [1993] and Maier [1983] are texts devoted exclusively to the relational data model. The original definition of relational algebra is in Codd [1970]; that of tuple relational calculus is in Codd [1972]. A formal proof of the equivalence of tuple relational calculus and relational algebra is in Codd [1972].

Several extensions to the relational calculus have been proposed. Klug [1982] and Escobar-Molano et al. [1993] describe extensions to scalar aggregate functions. Extensions to the relational model and discussions of incorporation of null values in the relational algebra (the RM/T model), as well as outer joins, are in Codd [1979]. Codd [1990] is a compendium of E. F. Codd's papers on the relational model. Outer joins are also discussed in Date [1993b]. The problem of updating relational databases through views is addressed by Bancilhon and Spyratos [1981], Cosmadakis and Papadimitriou [1984], Dayal and Bernstein [1978], and Langerak [1990]. Section 14.5 covers materialized view maintenance, and references to literature on view maintenance can be found at the end of that chapter.

© The McGraw–Hill Companies, 2001

PART 2

Relational Databases

A relational database is a shared repository of data. To make data from a relational database available to users, we have to address several issues. One is how users specify requests for data: Which of the various query languages do they use? Chapter 4 covers the SQL language, which is the most widely used query language today. Chapter 5 covers two other query languages, QBE and Datalog, which offer alternative approaches to querying relational data.

Another issue is data integrity and security; databases need to protect data from damage by user actions, whether unintentional or intentional. The integrity maintenance component of a database ensures that updates do not violate integrity constraints that have been specified on the data. The security component of a database includes authentication of users, and access control, to restrict the permissible actions for each user. Chapter 6 covers integrity and security issues. Security and integrity issues are present regardless of the data model, but for concreteness we study them in the context of the relational model. Integrity constraints form the basis of relational database design, which we study in Chapter 7.

Relational database design—the design of the relational schema—is the first step in building a database application. Schema design was covered informally in earlier chapters. There are, however, principles that can be used to distinguish good database designs from bad ones. These are formalized by means of several "normal forms," which offer different tradeoffs between the possibility of inconsistencies and the efficiency of certain queries. Chapter 7 describes the formal design of relational schemas.